

MÓDULO 12: Control de Versiones en la Nube, Colaboración en GitHub y CI/CD con GitHub Actions

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 1 hora y 30 minutos

Módulo: 12 - Git Remoto, Repositorios Personalizados

(`<user_name>_PYTHON_CIENCIA_ABIERTA_2`), GitHub Workflows, Licencias Abiertas & Continuous Integration (CI/CD)

1. Objetivos de Aprendizaje

- **Gestión de Repositorios Personalizados en la Nube:** Crear repositorios remotos en GitHub con el patrón de nombre de usuario (`<user_name>_PYTHON_CIENCIA_ABIERTA_2`), vincularlos con el paquete local generado por la plantilla **pyOpenSci** (`PYTHON_CIENCIA_ABIERTA_2`) y sincronizar versiones.
- **Privacidad y Licencias Abiertas:** Comprender la diferencia entre repositorios Públicos y Privados, proteger secretos (`.env` , claves API) mediante `.gitignore` y seleccionar licencias científicas (MIT, Apache 2.0, GPL).
- **Flujo de Trabajo Colaborativo (*Feature Branch Workflow*):** Trabajar con ramas locales (`git checkout -b`), realizar commits estructurados y enviar ramas al remoto (`git push`).
- **Revisiones de Código y Pull Requests (PR):** Proponer cambios, revisar diferencias (*diffs*) y fusionar código (*Merge*) mediante la interfaz web de GitHub.
- **Integración Continua (CI/CD) con GitHub Actions:** Entender el funcionamiento de los flujos automatizados de trabajo en la nube (`.github/workflows/test.yml`) para ejecutar `pytest` de forma automática tras cada commit.

2. Control de Versiones en la Nube, Privacidad y Licencias Abiertas

Publicar nuestro código en repositorios remotos como **GitHub** es una pieza angular de la **Ciencia Abierta**. Facilita que otros grupos de investigación auditen, reproduzcan y

reutilicen nuestros métodos computacionales.

Repositorios Públicos vs. Privados

- **Repositorio Privado:** El código solo es accesible por ti y por los colaboradores explícitamente invitados. Es ideal para las fases iniciales de un proyecto, o casi obligatorio antes de la publicación oficial del artículo científico.
- **Repositorio Público:** El código es visible para toda la comunidad internacional. Fomenta las citas científicas, la transparencia y la colaboración.

💡 **Consejo Pedagógico:** Puedes comenzar tu investigación en un repositorio privado en GitHub y, una vez aceptado el manuscrito en la revista científica, cambiar su visibilidad a **Público** con un solo clic en la llamada **zona de peligro** de GitHub.

Protección de Secretos y Datos Sensibles (`.gitignore`)

Nunca debemos subir al repositorio remoto:

1. **Archivos de entorno y credenciales (`.env`):** Claves de API, contraseñas de bases de datos o tokens de acceso a repositorios institucionales.
2. **Datos con restricciones de privacidad:** Datos personales de pacientes, muestras sensibles no anonimizadas, conjuntos de datos masivos, datos sometidos a NDA, etc...MUCHO CUIDADO!!!!!!
3. **Archivos generados automáticamente:** Entornos virtuales (`venv/`), carpetas de caché (`__pycache__`) o binarios de compilación.

Para ello, verificamos que nuestro archivo `.gitignore` incluya estas reglas antes del primer commit.

Licencias de Código Abierto

Un repositorio público sin licencia se considera con *"todos los derechos reservados"*, lo que impide legalmente que otros científicos reutilicen tu software. Las licencias más comunes en Ciencia Abierta son:

- **MIT License:** Permisiva y muy sencilla. Permite a cualquiera reutilizar, modificar y distribuir tu código libremente, siempre que mantengan la nota de copyright.
- **Apache License 2.0:** Permisiva, pero incluye una concesión explícita de derechos de patente.
- **GNU GPLv3:** Licencia *Copy/left*. Exige que cualquier obra derivada sea publicada también bajo la misma licencia libre GPL.

3. Paso 1: Creación del Repositorio Remoto en GitHub con tu Nombre de Usuario

Para subir el paquete que creamos en el Módulo 11 con la plantilla de **pyOpenSci** (`PYTHON_CIENCIA_ABIERTA_2`), crearemos un repositorio remoto en GitHub añadiendo tu nombre de usuario para diferenciarlo en la comunidad.

Pasos en la Web de GitHub:

1. Inicia sesión en tu cuenta de [GitHub](#).
2. Haz clic en el botón verde **"New"** o dirígete a `https://github.com/new`.
3. Completa los campos:
 - **Repository name:** `<user_name>_PYTHON_CIENCIA_ABIERTA_2` (Sustituye `<user_name>` por tu nombre de usuario de GitHub, por ejemplo: `guslicem_PYTHON_CIENCIA_ABIERTA_2`).
 - **Description:** Librería de ejemplo para el empaquetado y CI/CD en Ciencia Abierta.
 - **Visibilidad:** Selecciona **Private** (o **Public** según prefieras).
 - **⚠ IMPORTANTE: NO marques "Add a README file", "Add .gitignore" ni "Choose a license". El repositorio remoto debe crearse 100% vacío** porque nuestro paquete local ya contiene esos archivos generados por `copier`.
4. Haz clic en **"Create repository"**.
5. Copia la URL HTTPS de tu repositorio (ejemplo:
`https://github.com/guslicem/guslicem_PYTHON_CIENCIA_ABIERTA_2.git`).

4. Paso 2: Sincronización del Paquete Local con el Repositorio Remoto

Ahora abrimos nuestra terminal integrada y vinculamos nuestra carpeta local con la dirección remota personalizada de GitHub.

Comandos de Configuración e Inicialización:

```
# 1. Nos aseguramos de estar en la carpeta de nuestro paquete
cd NOTEBOOKS/PYTHON_CIENCIA_ABIERTA_2

# 2. Configurar nuestra identidad global en Git (si no lo habíamos
hecho antes)
git config --global user.name "Tu Nombre o Usuario"
git config --global user.email "tu_email@csic.es"

# 3. Inicializar el repositorio Git local (si no estuviera
inicializado)
git init
```

4. Asegurarnos de que la rama principal se llama 'main'

```
git branch -M main
```

5. Vincular nuestro repositorio local con la URL de tu repositorio remoto en GitHub:

Sustituye <user_name> por tu nombre de usuario real de GitHub

```
git remote add origin
```

```
https://github.com/<user_name>/<user_name>_PYTHON_CIENCIA_ABIERTA_2.git
```

Ejemplo real para el usuario 'guslicem':

git remote add origin

```
https://github.com/guslicem/guslicem_PYTHON_CIENCIA_ABIERTA_2.git
```

6. Añadir todos los archivos al área de preparación (staging)

```
git add .
```

💡 Tras la incorporación de los cambios. El "hacer la foto", debemos ponerle una etiqueta a la foto, es lo que se llama el mensaje de **"commit"** Este estándar ayuda a que el historial de Git sea limpio, legible y fácil de auditar de un vistazo:

Prefijos standard en los **commits**

- feat: Nueva característica/funcionalidad para el usuario. feat: añadir función de exportación a LaTeX
- fix: Corrección de un fallo o error en el código. fix: corregir error de cálculo de medias en nulos
- docs: Cambios exclusivamente en la documentación (.md, docstrings). docs: actualizar instrucciones en el README.md
- chore: Tareas de mantenimiento o configuración del repositorio. chore: actualizar .gitignore y desindexar carpetas
- refactor: Reorganizar código sin cambiar su comportamiento. refactor: simplificar bucle de filtrado en Pandas
- test: Añadir o modificar pruebas unitarias (pytest). test: añadir pruebas para la clase Persona

7. Crear el primer commit oficial

```
git commit -m "feat: inicializar paquete estandarizado pyOpenSci con tests"
```

8. Subir el código a GitHub y vincular la rama main por defecto (-u)

```
git push -u origin main
```

9. Navega ahora a la web de tu repo y observa qué ha pasado.

En mi caso este es el acceso:

[REPOSITORIO CREADO](#)

5. Paso 3: Flujo de Trabajo con Ramas (*Feature Branch Workflow*)

¿Qué es una Rama (*Branch*) en Git?

Una **rama** en Git es simplemente una **línea de desarrollo independiente y paralela**. Imagina que tu repositorio es un árbol:

- **La rama principal (`main`)**: Representa el tronco del árbol. Es la versión oficial, estable, probada y lista para producción o publicación de tu proyecto.
- **Una rama secundaria (`feature/...`)**: Es una ramificación que nace a partir de `main` y crea una copia exacta de tu código en un espacio de trabajo aislado.

Cualquier cambio, experimento o error que introduzcas en una rama secundaria **no afectará para nada al código de `main`** hasta que decidas fusionarlo explícitamente.

¿Por qué debemos trabajar con ramas en Ciencia Abierta?

1. **Protección del Código Estable**: Mantiene la rama `main` siempre limpia y en perfecto estado funcional.
2. **Experimentación Sin Riesgos**: Te permite desarrollar nuevas funciones o probar ideas arriesgadas sin miedo a estropear el trabajo de tus compañeros.
3. **Revisión por Pares (Code Review)**: Facilita que otros investigadores del equipo revisen tus cambios antes de incorporarlos definitivamente al proyecto.

Paso 3.1: Crear y Cambiar a una Rama de Trabajo Local

Desde la terminal (dentro de la carpeta de tu paquete `PYTHON_CIENCIA_ABIERTA_2` y con `hatch shell` activo):

```
# Crear y cambiar a una nueva rama de desarrollo para la validación
de edad
git checkout -b feature/validacion-edad-persona

# Comprobar la lista de ramas y verificar cuál está activa (marcada
con *)
git branch
```

Paso 3.2: Implementar la Nueva Regla en la Clase Persona

Abrimos el archivo de nuestra clase (src/python_ciencia_abierta_2/gus_class.py) e incorporamos la restricción de que el personal del CSIC debe tener al menos 18 años. Busca el constructor (`__init__`) y debajo añade:

```
def __init__(self, nombre: str, edad: int, instituto: str = None,
servicio: str = None):
    # Validación de edad mínima para personal investigador/técnico
    if edad < 18:
        raise ValueError("La edad debe ser mayor o igual a 18 años
para ser personal del CSIC.")
```

Paso 3.3: Añadimos una función de test.

Este test comprobará que al intentar instanciar a un menor de 18 años lanza la excepción esperada. Para ello, en tests/unit/test_persona.py añade esta nueva funcionalidad.

```
def test_persona_error_menor_edad():
    """Verifica que se lanza ValueError al intentar registrar
personal menor de 18 años."""
    msg = "Verificando restricción de edad mínima (< 18 años) en el
CSIC..."
    print(msg)
    with pytest.raises(ValueError, match="La edad debe ser mayor o
igual a 18 años"):
        Persona(nombre="Estudiante Becario", edad=17,
instituto="IMSE")
```

Paso 3: Instalar el paquete en modo editable

Una vez que has configurado tu paquete, puedes instalarlo en modo editable para facilitar el desarrollo. Esto te permite modificar el código fuente sin tener que reinstalar el paquete cada vez.

Instalación editable con Hatch

Navega al directorio raíz del paquete y ejecuta los tests con pytest

Observa como hatch ha creado un entorno virtual propio para tu paquete lo que te va a permitir trabajar con él sin provocar interacciones no-deseadas con otros paquetes en nuestro sistema.

6. Paso 4: Pull Requests (PR) y Revisión de Código en GitHub

Un **Pull Request (PR)** es una solicitud formal para fusionar el código de nuestra rama secundaria (`feature/nuevos-tests-persona`) en la rama principal (`main`).

Flujo de Trabajo de un Pull Request:

1. **Apertura del PR:** Al hacer `git push` de tu rama, entra a la página de tu repositorio en GitHub
(`https://github.com/<user_name>/<user_name>_PYTHON_CIENCIA_ABIERTA_`
2. Haz clic en el botón destacado **"Compare & pull request"**.
3. **Descripción:** Escribe un título descriptivo y explica qué cambios introduce tu PR.
4. **Revisión de Código (Diffs):** Los colaboradores o revisores del grupo pueden examinar línea por línea los cambios realizados (líneas verdes `+` agregadas y líneas rojas `-` eliminadas).
5. **Aprobación y Fusión (Merge):** Una vez aprobado, se pulsa el botón **"Merge pull request"** y los cambios pasan a formar parte oficial de la rama `main`.
6. **Sincronización Local:** En tu máquina local, regresas a `main` y descargas las novedades:
`git checkout main`
`git pull origin main`

7. Paso 5: Integración Continua (CI/CD) con GitHub Actions

La **Integración Continua (CI)** consiste en automatizar la compilación, verificación y ejecución de pruebas unitarias de nuestro proyecto cada vez que alguien envía nuevo código a GitHub.

¿Cómo funciona GitHub Actions en nuestro Paquete?

Cuando generamos la plantilla con **pyOpenSci**, `copier` creó automáticamente una carpeta oculta `.github/workflows/` con el archivo `test.yml`:

```
PYTHON_CIENCIA_ABIERTA_2/
├── .github/
│   └── workflows/
│       └── test.yml
```

← Configuración del flujo de automatización de Pytest

⚙️ ¿Qué hace el archivo `test.yml` en la Nube?

Cada vez que haces `git push` o abres un Pull Request:

1. **GitHub arranca una máquina virtual remota (Ubuntu Linux) en la nube.**
2. **Instala Python (por ejemplo, versión 3.10, 3.11, 3.12).**
3. **Instala las dependencias de tu proyecto (`pyproject.toml`).**
4. **Ejecuta automáticamente la orden `pytest` .**
5. Si todos los tests pasan, GitHub asigna una marca verde 🟢 (**PASSED**) a tu commit.
Si algún test falla, muestra una cruz roja 🛑 (**FAILED**) y envía una alerta por correo electrónico.

⚙️ Vamos a editar el archivo `test.yml` . Para ello:

1. **Navega al directorio `.github/workflows/test.yml` del repositorio**
2. **Edita el fichero y busca:** `strategy: fail-fast: false matrix: os: [ubuntu-latest, macos-latest, windows-latest] python-version: ["3.10", "3.13"]`
3. **Haz que el proyecto se testee para las versiones de python 3.11,3,12 y 3.13 cambiando la línea `python-version` asi:** `python-version: ["3.11", "3.12", "3.13"]`

🇺🇹 Badges de Estado en el `README.md`

Podemos incluir un distintivo visual (*Badge*) en la parte superior de nuestro `README.md` que muestra el estado de salud del proyecto en tiempo real:

```
![Tests]
(https://github.com/guslicem/guslicem\_PYTHON\_CIENCIA\_ABIERTA\_2/actions,
```

8. Ejercicios Prácticos y Autoevaluación

Ejercicio Práctico 1: Simulación Completa de Flujo de Trabajo en GitHub

1. Crea una rama local llamada `feature/actualizar-readme` .
2. Modifica el archivo `README.md` de tu paquete añadiendo una breve descripción de tu proyecto de investigación.
3. Haz el `commit` y sube la rama a GitHub (`git push -u origin feature/actualizar-readme`).
4. Abre un Pull Request en GitHub para tu repositorio `<user_name>_PYTHON_CIENCIA_ABIERTA_2` , verifica que el flujo de **GitHub Actions** ejecute las pruebas en la nube y aprueba el PR haciendo *Merge*.

5. Vuelve a tu terminal local y actualiza tu rama `main` ejecutando `git checkout main` y `git pull`.

Ejercicio Práctico 2: Auditoría de Seguridad de Secretos

Comprueba el estado del archivo `.gitignore` de tu repositorio y confirma que la regla `.env` esté presente para evitar subir claves privadas a la nube.

```
In [ ]: # Solución Interactiva Ejercicio 2: Verificación de Reglas de Ignorado en Py
import os

gitignore_path = os.path.join(".", ".gitignore")

if os.path.exists(gitignore_path):
    with open(gitignore_path, "r", encoding="utf-8") as f:
        reglas = [line.strip() for line in f if line.strip() and not line.st

    print("🔒 Reglas activas detectadas en .gitignore:")
    for r in reglas:
        print(f" - {r}")

    print("")
    if any(".env" in r for r in reglas):
        print("✅ ¡Seguridad Confirmada! La regla .env está activa para prot
    else:
        print("⚠️ ALERTA: No se detectó la regla .env en .gitignore.")
else:
    print(f"❗ El archivo .gitignore no se encuentra en la ruta esperada: {g
```



Cuestionario de Auto-evaluación y Participación

¡Pon a prueba los conocimientos adquiridos en este módulo! Completa este breve cuestionario interactivo de 5 minutos en Google Forms:

👉 [Cuestionario Módulo 12: Control de Versiones en GitHub y CI/CD](#)
